Termohygrometr na Raspberry PI

Ročníková práce z předmětu PSS

Jan Čaloun

C3c

Zadání práce

Cílem tohoto projektu je navrhnout a zkonstruovat funkční termohygrometr na Raspberry Pi, který bude ukazovat data na webu. Projekt se skládá z hardwarové a softwarové části a musí splňovat:

- Měření aktuální teploty a vlhkosti vzduchu pomocí fyzického senzoru připojeného k Raspberry Pi.
- Pravidelné ukládání naměřených dat a historie do databáze.
- Vytvoření responzivního webového rozhraní.

Obsah

Zadání práce 2
Obsah 3
Úvod 4
Rozbor 5
Výběr hardwaru 5
Výběr softwaru 5
Realizace 8
1. Fyzické zapojení hardwaru 8
2. Příprava databáze InfluxDB 9
3. Programování logiky 9
4. Tvorba aplikačního rozhraní (API) pomocí frameworku Flask 11
5. Automatické spouštění skriptů (systemd) 11
6. Vytvoření uživatelského rozhraní 12
7. Zpracování dat a vizualizace 13
8. Zajištění vzdáleného přístupu pomocí Ngrok 14
Závěr 15
Seznam obrázků 16
Použitá literatura a zdroje 17

Úvod

Části chytré domácnosti a IoT (internet věcí) se používají naprosto běžně. Sledování teploty a vlhkosti v pokoji se hodí, protože moc suchý nebo naopak moc vlhký vzduch zhoršuje soustředění i kvalitu spánku.

Cílem projektu bylo postavit vlastní chytrý senzor, který tyto hodnoty dlouhodobě udrží a zobrazí na webu. Jako základ jsem použil senzor DHT11 a mikropočítač Raspberry Pi.

Tato dokumentace vyobrazuje, jak celý projekt vznikal. V první části stručně představím použitý hardware a software. Poté v hlavní části dokumentace popisuji postup krok za krokem, od zapojení drátků, až po tvorbu webu. Práce slouží jako návod, jak si podobné zařízení postavit od nuly.

Rozbor

Výběr hardwaru

Zvolil jsem **Raspberry Pi zero 2wh** jako můj mikropočítač kvůli skvělému poměru cena a výkon. Tato verze Raspberry má rychlý čtyřjádrový procesor, 512 MB operační paměť RAM, což je dostatečné na plynulý běh databáze a webu.

Další klíčový faktor byla velmi nízká spotřeba energie, což je u zařízení běžícího 24 hodin denně zásadní. Verze WH obsahuje integrovaný Wi-Fi modul a předem připájené GPIO piny, což výrazně usnadnilo připojení senzoru.

K tomuto mikropočítači jsem zvolil paměťovou microSD kartu o velikosti 32 GB. Tu jsem vybral kvůli její kompatibilitě s mikropočítačem, výkoností, a hlavně kvůli její výdrži při neustálém zapisování dat.

Na samotné měření jsem vybral digitální **senzor DHT11**. Jeho hlavní výhoda je nízká pořizovací cena a velká podpora od komunity, což znamená snadnou dostupnost softwarových knihoven.

Senzor je napojený s mikropočítačem, od kterého dostává napájecí napětí 3,3V. Ačkoliv má tento senzor nižší přesnost měření, tak to pro domácí měření stačí.

Výběr softwaru

1. **Operační systém a hlavní jazyk**

Jako operační systém pro mikropočítač byl zvolen **Debian GNU/Linux 13 (verze Trixie).** Tato linuxová distribuce byla vybrána kvůli své bezkonkurenční stabilitě a minimálními nároky na hardware.

Pro tvorbu veškerých logický programů – čtení dat ze senzoru, jejich ukládání a poskytování webového rozhraní – byl vybrán programovací jazyk **Python**. Byla to jasná volba díky obrovskému množství hotových a spolehlivých knihoven.

2. **Databáze – InfluxDB**

Místo klasických relačních databází (jako MySQL nebo SQLite) byla pro ukládání historie měření zvolena databáze **InfluxDB**. Je to tzv. Time-Series Database, takže databáze optimalizovaná přímo pro ukládání časových dat.

S tím, že termohygrometr generuje nepřetržitý proud dat (vlhkost a teplota) vázaných na konkrétní časový údaj, je InfluxDB tou nejlepší volbou. Zároveň má rychlí zápis a jednoduché dotazování na časové úseky.

3. Frontend a vizualizace

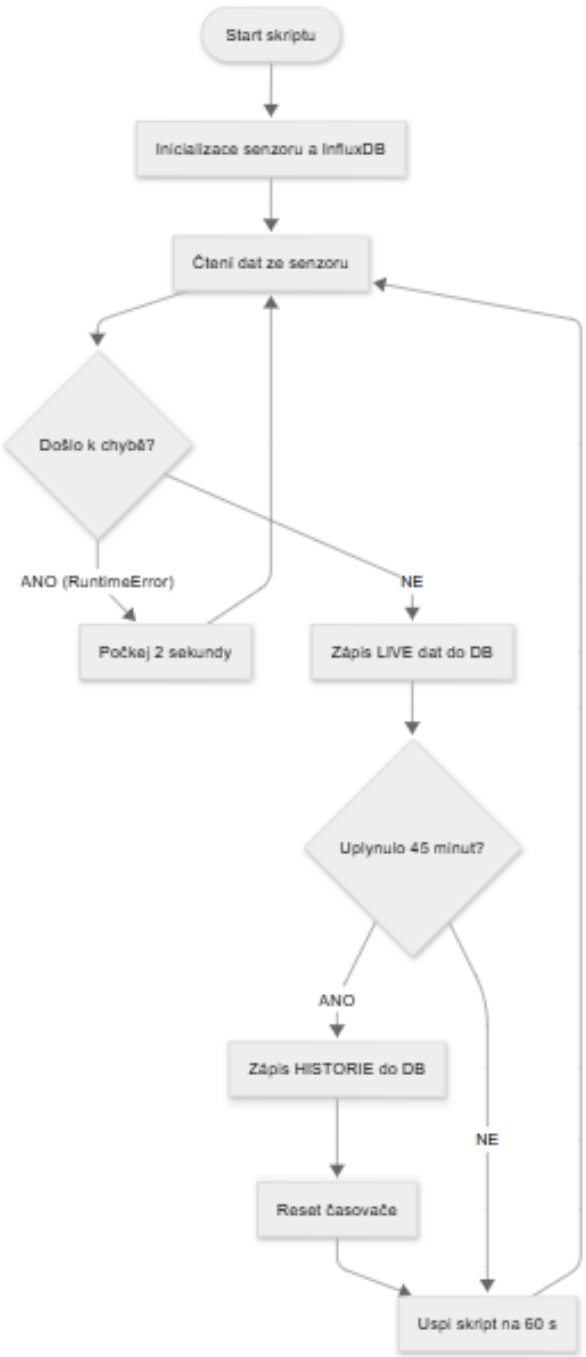
Web byl vytvořen pomocí standartních webových technologií **HTML5, CSS3 a JavaScriptu**. Pro zajištění responzivity byl využit CSS Grid a Flexbox.

Na vykreslování historie teploty a vlhkosti do grafů byla použita volně přístupná knihovna **Chart.js**. Byla vybrána kvůli své nízké datové náročnosti, jednoduchému vykreslování na element <*canvas*> a nativní podporu pro více nezávislých os Y.

4. Zajištění přístupu zvenčí

Aby bylo možné sledovat web odkudkoliv z internetu, bez toho, aby bylo nutné platit za veřejnou IP adresu nebo složitě nastavovat Port Forwarding, což by bylo nebezpečné, byl využit nástroj **Ngrok**.

Vytváří bezpečný šifrovaný tunel mezi lokálním portem na Raspberry Pi a veřejným koncem na serverech společnosti Ngrok. Pomocí toho je web přístupný přes standartní HTTPS protokol z jakéhokoliv zařízení připojeného k internetu.



Obrázek 1- Diagram algoritmu pro ukládání dat

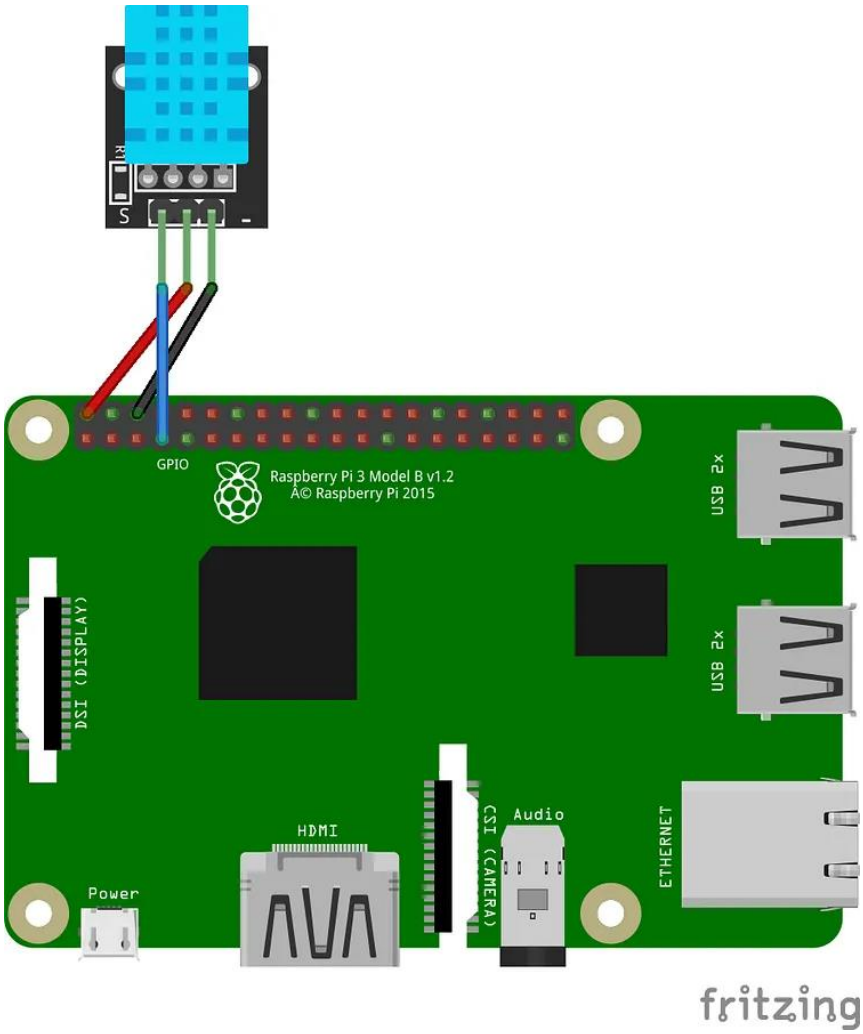
Realizace

1. Fyzické zapojení hardwaru

Prvním krokem bylo fyzické propojení senzoru DHT11 s mikropočítačem. Na propojení byly využity standartní vodiče typu samice-samice (jumper kabely), které byly nasunuty na připájené GPIO piny mikropočítače.

Senzor DHT11 používá tři piny, které byly zapojeny takto:

- **VCC** (Napájení): Připojeno na pin č.1 (3,3V) na mikropočítači.
- **GND** (Zem): Připojeno na pin č.6 (Ground).
- **DATA** (Datový signál): Připojeno na pin č.7 (GPIO 4).



Obrázek 2 - Blokové schéma DHT11--Raspberry Pi

2. Příprava databáze InfluxDB

K instalaci operačního systému byl použit nástroj Raspberry Pi Imager. Tento program usnadnil konfiguraci celého OS. Ať už nastavení přihlašovacích údajů k lokální Wi-Fi a povolení vzdáleného přístupu přes protokol SSH. Po konfiguraci byl na paměťovou microSD kartu nahrán obraz vybraného systému (Raspberry Pi OS Lite na bázi Debian GNU/Linux).

Po úspěšném spuštění systému a aktualizaci základních balíčků bylo nutné zajistit úložiště pro naměřená data. Byla nainstalována time-series databáze **InfluxDB**, která běží jako služba na pozadí mikropočítače a naslouchá na portu 8086. Přes příkazový řádek byla poté vytvořená databáze **moje_senzory**, do které hlavní program zapisuje všechny hodnoty.

3. Programování logiky

Pro načítání dat ze senzoru a následné ukládání do databáze byl vytvořen skript v jazyce Python. Před spuštěním bylo nutné doinstalovat knihovny **adafruit-circuitpython-dht** pro komunikaci se senzorem a **influxdb** pro spojení s databází.

Hlavní část kódu tvoří smyčka (**while True**), která provádí neustálý běh programu na pozadí. Skript má logiku dvojitého zápisu, aktuální data pro okamžité zobrazení, které je přepisováno každou minutu, zatímco historická data pro grafy ukládá jedno měření každých 45 minut.

Kód je uschován v bloku pro odchytávání výjimek (**try/except**). Pokud senzor selže, program zachytí chybu (**RuntimeError**), počká dvě sekundy a pokusí se o čtení znovu, čímž je zaručená stabilita zápisů a ochrana proti jakýmkoliv chybám.

```python
# Proměnná pro sledování času posledního zápisu do historie
posledni_velky_log = 0
while True:
    try:
        # Přečtení hodnot ze senzoru a zjištění aktuálního času
        temperature_c = dhtDevice.temperature
        humidity = dhtDevice.humidity
        aktualni_cas = time.time()

        # Ukládáme do samostatného měření, abychom nezahlcovali grafy historie
        json_live = [
            {
                "measurement": "teplota_live",
                "tags": {"umisteni": "pokoj"},
                "fields": {
                    "teplota": float(temperature_c),
                    "vlhkost": float(humidity)
                }
            }
        ]
        client.write_points(json_live)

        # 2700 sekund = 45 minut. Pokud tento čas uplynul, zapíšeme bod do historie.
        if aktualni_cas - posledni_velky_log >= 2700:
            json_history = [
                {
                    "measurement": "senzor_historie",
                    "tags": {"umisteni": "pokoj"},
                    "fields": {
                        "teplota": float(temperature_c),
                        "vlhkost": float(humidity)
                    }
                }
            ]
            client.write_points(json_history)

            # Vynulování časovače - uložíme si čas tohoto posledního velkého zápisu
            posledni_velky_log = aktualni_cas

    # 3. OŠETŘENÍ CHYB (Error Handling)
    except RuntimeError as error:
        # počkáme 2 sekundy a smyčka se bez pádu programu zopakuje.
        time.sleep(2.0)
        continue

    except Exception as error:
        # Při jakékoliv jiné kritické chybě bezpečně odpojíme senzor a program ukončíme
        dhtDevice.exit()
        raise error
    # Uspání skriptu na 60 sekund před dalším měřením
    time.sleep(60.0)
```

Obrázek 3 - Hlavní smyčka pro měření

4. Tvorba aplikačního rozhraní (API) pomocí frameworku Flask

Aby bylo možné předávat naměřená data z databáze do webového prohlížeče (frontend), bylo nutné vytvořit prostředníka tzv. lokální aplikační rozhraní (API). Na toto byl použit postačující webový framework **Flask** pro jazyk Python.

Flask server běží na pozadí a naslouchá lokálnímu portu. Je nastaven jako backhendová služba, jakmile příjme http požadavek z webového rozhraní, odešle dotaz do databáze InfluxDB a získá nejnovější "Live" data nebo historická data pro grafy, které zformátuje a odešle klientovy ve formátu **JSON**.

5. Automatické spouštění skriptů (systemd)

Pro neustálý chod obou skriptů bylo potřeba zajistit, aby se sběr dat i Flask API spouštěly automaticky ihned po zapnutí mikropočítače (například po výpadku proudu).

Byl využit správce služeb operačního systému Linux – **systemd**. V systémovém adresáři */etc/systemd/systém/* byly vytvořeny vlastní konfigurační soubory s příponou **.service** (jeden pro sbírání dat a druhý pro API). Díky tomuto řešení běží skripty na pozadí jako systémové služby. Velkou výhodou je funkce automatického restartu (Restart=always), pokud by došlo k pádu Python skriptu.

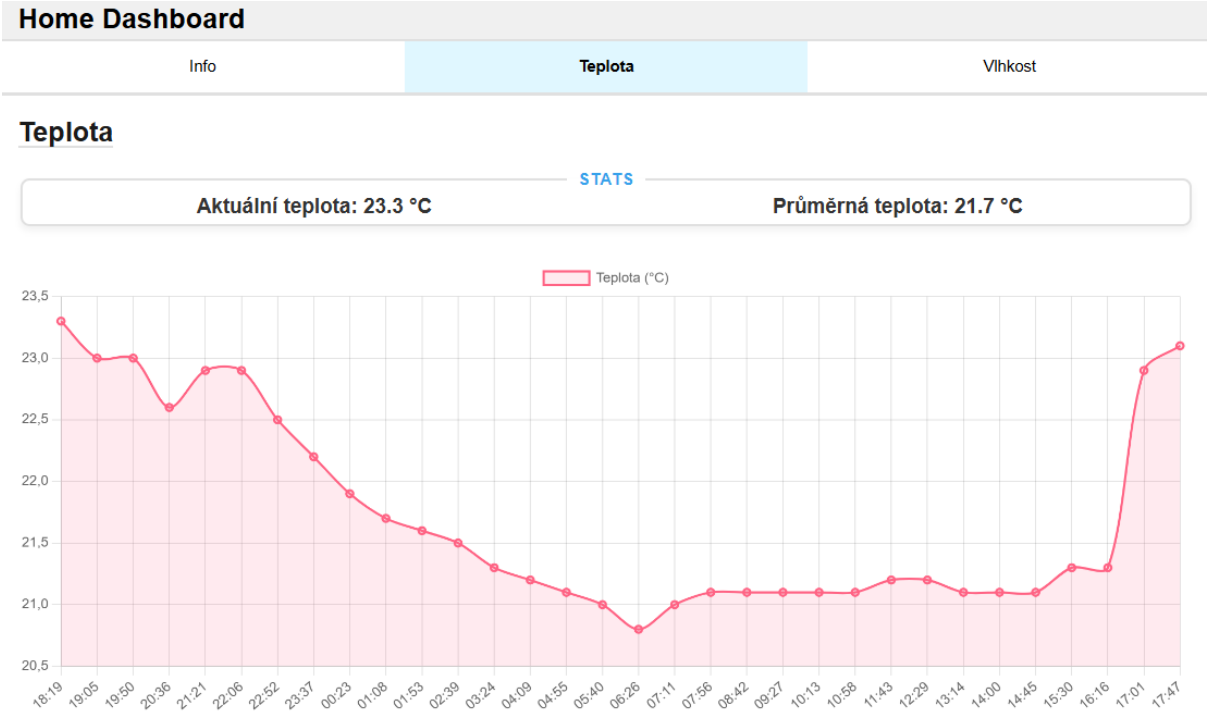
```
[Unit]
Description = DHT11 Sensor Logging to InfluxDB
# Spustit až po naběhnutí sítě a Dockeru
After = network.target docker.service

[Service]
# Příkaz pro spuštění a pracovní složka
ExecStart = /usr/bin/python3 /home/caloun/PSS_project/sensor.py
WorkingDirectory = /home/caloun/PSS_project

# Automatický restart služby při pádu (s prodlevou 10 s)
Restart = always
RestartSec = 10

# Běží pod běžným uživatelem (bezpečnost)
User = caloun

[Install]
# Zajistí start služby ihned po zapnutí Raspberry Pi
WantedBy = multi-user.target
```

Obrázek 4 - Spouštěcí servis pro měření dat

6. Vytvoření uživatelského rozhraní

Vizuální část projektu (dashboard) byla vytvořena pomocí webových technologií **HTML5** a **CSS3.** Cílem bylo vytvořit jednoduché a moderní rozhraní, které na jedné obrazovce zobrazí jak aktuální stav klimatu v místnosti, tak jeho historii.

Na rozvržení prvků byl použit novodobý **CSS Grid** s **Flexboxem.** To umožnilo vytvořit responzivní mřížku, která se mění podle velikosti obrazovky. Na monitoru se grafy a aktuální hodnoty skládají vedle sebe, zatímco na telefonu se seřadí pod sebe.



Obrázek 5 - Webový dashboard s grafem teploty (tablet)

7. Zpracování dat a vizualizace

Aby webová stránka zobrazovala reálná data bez nutnosti aktualizace stránky, byla použita logika v čistém JavaScriptu. Skript využívá asynchronní funkci **Fetch API**, pomocí které se na pozadí dotazuje vytvořeného Flask serveru na data.

Na vykreslování historie teploty a vlhkosti byla využita open-source knihovna **Chart.js**. Ta data získaná z API transformuje a poté vykresluje na HTML element <*canvas*>**.** Kvůli tomu, že teplota a vlhkost mají rozdílné jednotky a rozsahy, byly v nastavování grafu vytvořeny dvě nezávislé osy Y, což zajišťuje přehlednost obou křivek.

```
// Univerzální funkce pro vykreslení grafu pomocí Chart.js
function vykresliGraf(canvasClass, label, labels, data, barva) {
    const canvas = document.querySelectorAll(`.${canvasClass}`);
    if (canvas.length === 0) return;

    canvas.forEach(canvas => {

        // Zničení předchozí instance grafu zamezí překrývání při obnově dat
        const existujiciGraf = Chart.getChart(canvas);
        if (existujiciGraf) {
            existujiciGraf.destroy();
        }
        new Chart(canvas, {
            type: 'line',
            data: {
                labels: labels,
                datasets: [{
                    label: label,
                    data: data,
                    borderColor: barva,
                    // Přidání hexadecimálního '22' vytvoří poloprůhlednou výplň pod křivkou
                    backgroundColor: barva + '22',
                    borderWidth: 2,
                    tension: 0.3,
                    fill: true,
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,// Umožní grafu měnit výšku podle CSS Flexboxu/Gridu
                scales: {
                    y: {
                        // Osa Y nezačíná na nule, ale přizpůsobí se hodnotám (např. 20-25 °C)
                        beginAtZero: false
                    }
                }
            }
        })
    });
}
```

Obrázek 6 - Univerzální funkce pro vykreslování dat

8. Zajištění vzdáleného přístupu pomocí Ngrok

V tomto kroku byl zajištěn přístup na web z vnější sítě pomocí nástroje **Ngrok**. Tento software vytváří šifrovaný tunel mezi lokálním portem Raspberry Pi a severy společnosti Ngrok. Použití spočívalo pouze ve stažení linuxového klienta, spárování s účtem pomocí speciálního tokenu a spuštění služby naslouchající na portu webového serveru.

Příkaz pro spuštění tunelu: ***ngrok http 5000***

Závěr

Cílem tohoto projektu bylo zkonstruovat a realizovat funkční systém pro měření, ukládání a vizualizaci teploty a vlhkosti. Tento cíl byl splněn. Výsledkem je samostatně fungující IoT zařízení postavené na Raspberry Pi, které neustále sbírá data ze senzoru DHT11 a bezpečně je ukládá do lokální databáze.

V průběhu se podařilo správně propojit několik technologií. Backhendová část v Pythonu zajišťuje sběr dat a díky frameworku Flask dodává REST API. Frontendová část zobrazuje data, na přehledným a responzivním webu s použitím asynchronního vykreslování grafů přes knihovnu Chart.js. Zprovoznění nástroje Ngrok dodalo dosah do reálného světa, protože je dashboard dostupný odkudkoliv z internetu.

Během celého projektu bylo podstatné vyřešit několik problémů, zejména občasné chyby při čtení ze senzoru DHT11. Tento problém byl vyřešen a eliminován pomocí softwarového odchytávání výjimek a opakovaného měření. Celý projekt je zároveň spravován pomocí systemd, takže nevyžaduje žádnou údržbu.

Seznam obrázků

Obrázek 1- Diagram algoritmu pro ukládání dat 7
Obrázek 2 - Blokové schéma DHT11--Raspberry Pi 8
Obrázek 3 - Hlavní smyčka pro měření 10
Obrázek 4 - Spouštěcí servis pro měření dat 11
Obrázek 5 - Webový dashboard s grafem teploty (tablet) 12
Obrázek 6 - Univerzální funkce pro vykreslování dat 13

Použitá literatura a zdroje

[1] *Setting Up A DHT11 Sensor On Your RASPBERRY PI — The Easy Way*. Online. 2021. Dostupné z: https://piaverous.medium.com/setting-up-dht11-temp-hum-sensor-with-your-raspberry-pi-b408c0d4490f. [cit. 2026-03-16].

[2] **ADAFRUIT INDUSTRIES.** *DHT Humidity Sensing on Raspberry Pi with GDocs Logging* [online]. Dostupné z: https://learn.adafruit.com/dht-humidity-sensing-on-raspberry-pi-with-gdocs-logging/python-setup. [cit. 2026-03-16].

[3] *Chart.js*. Online. 2025. Dostupné z: https://www.chartjs.org/docs/latest/. [cit. 2026-03-16].

[4] *Creating a Linux service with systemd*. Online. 2017. Dostupné z: https://medium.com/@benmorel/creating-a-linux-service-with-systemd-611b5c8b91d6. [cit. 2026-03-16].